

# Recognizing temporary construction site objects using CLIP-based few-shot learning and multi-modal prototypes

Yuanchang Liang<sup>a,\*</sup>, Prahlad Vadakkepat, PhD<sup>a</sup>, David Kim Huat Chua, PhD<sup>b</sup>, Shuyi Wang<sup>b</sup>, Zhigang Li<sup>b</sup>, Shuxiang Zhang<sup>b,c</sup>

<sup>a</sup> Department of Electrical and Computer Engineering, National University of Singapore, S117583, Singapore

<sup>b</sup> Department of Civil and Environmental Engineering, National University of Singapore, S117576, Singapore

<sup>c</sup> College of Civil Engineering, Tongji University, 200092 Shanghai, China

## ARTICLE INFO

### Keywords:

Construction sites  
CLIP  
Few-shot image classification (FSIC)  
Computer vision  
Robots

## ABSTRACT

Visual understanding of temporary on-site objects is essential for robots and project management in construction. Implementation of deep learning algorithms is challenging on construction sites due to high data annotation cost, demanding computational power, and lack of large-scale training datasets. Recognizing on-site temporary objects demands the algorithms to learn in a data-efficient way. To fill this gap, a Contrastive Language–Image Pre-training (CLIP)-based few-shot learning algorithm to recognize temporary objects with limited image samples is proposed. The study builds ImageNet-based Similarity Cache with inter-class similarity distribution. The proposed algorithm is evaluated on a newly created TOCS dataset and on the public SODA dataset. Compared with CLIP zero-shot algorithm, the classification accuracy improves from 23.17% to 73.09% with 16-shot learning on SODA, and from 58.33% to 83.33% with 1-shot learning on TOCS. The study indicates that few-shot learning with vision language models (VLM) is promising to improve visual intelligence on construction sites.

## 1. Introduction

AI-powered robotics is anticipated to play a significant role in the future of the Architecture, Engineering, and Construction (AEC) sector [1]. Incorporating robotic systems into construction offers benefits like decreased injuries, automation of repetitive tasks, and facilitating construction in challenging or dangerous environments, such as disaster relief and extraterrestrial construction [2]. Both academia and industry have directed their efforts towards developing autonomous construction robots capable of performing diverse tasks on construction sites, encompassing structural assembly, additive manufacturing, and finishing tasks [2]. Computer vision is one of the critical perception abilities for AI-enabled robots to understand the site environment. Objects on construction sites can be divided into structural elements and temporary objects. In contrast to structural components found on construction sites, temporary objects are positioned and relocated by workers as needed to meet specific requirements for a limited duration. The visual understanding of temporary items like materials, machinery, and tools aids in the automation of robotic construction for enhanced control. For

example, an automated material handling robot equipped with pick-and-place capabilities must visually discern target materials amidst cluttered construction sites. This visual understanding of temporary construction resources greatly influences safety, quality, speed, and profitability in construction. A typical example is the ability to detect workers' interactions with construction objects, enabling the monitoring of productivity and safety hazards on sites [4].

Understanding temporary objects visually entails robots categorizing objects by capturing images using digital cameras. Object classification, a fundamental computer vision task, assigns objects to predefined categories based on spectral characteristics, shape, texture, and spatial relationships with surrounding pixels [5]. In contemporary computer vision, Deep Learning (DL)-based models have achieved remarkable performance in tasks like image classification [6]. Despite high accuracy on public datasets, traditional computer vision algorithms demand hundreds or even thousands of images for robust neural network training. Annotated datasets are a significant challenge for deploying deep learning-based computer vision algorithms [4]. Implementing data-driven algorithms poses three primary challenges for recognizing

\* Corresponding author.

E-mail address: [e0572565@u.nus.edu](mailto:e0572565@u.nus.edu) (Y. Liang).

<https://doi.org/10.1016/j.autcon.2024.105542>

Received 19 October 2023; Received in revised form 5 June 2024; Accepted 7 June 2024

Available online 21 June 2024

0926-5805/© 2024 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

temporary objects on construction sites. Firstly, construction domain datasets are often small-scale and limited in sharing due to privacy concerns, making it impractical to have comprehensive datasets with various temporary objects and construction environment contexts. Secondly, the expensive time and labor involved in data annotation hinders the development of large-scale datasets for unique construction sites, disrupting workflow integration [3]. Thirdly, construction sites exhibit dynamic object movements, varying across temporal and spatial contexts [4]. Given neural networks' limited generalization ability, consistently recognizing novel objects throughout diverse construction project phases poses a challenge. Therefore, computer vision algorithms requiring fewer training samples and less training time are preferable for construction sites.

Few-shot image classification (FSIC) poses a significant challenge in identifying novel objects with limited image samples. While recent research has seen a surge in FSIC interest on public benchmarks, the construction domain has received scant attention. Typically, a few-shot image classification algorithm begins with training a base model on a large-scale dataset containing objects from base classes. Subsequently, the model is fine-tuned using a support set comprising only a few labeled examples of novel classes. Finally, the model's performance is evaluated on a test set containing novel classes. Previous FSIC research has focused on acquiring meta-knowledge from the base dataset, which significantly influences classification effectiveness [7]. However, our comprehension of how to establish an optimal base dataset remains limited. The requirement to train on a base dataset poses challenges for on-site learning by construction robots. Therefore, we propose training-free [8] CLIP-based [9] few-shot learning for on-site temporary object classification.

The paper proposes a CLIP-based few-shot learning algorithm for temporary object recognition on construction sites. The work has two major contributions. First, the study proposes an ImageNet-based Similarity Cache to enrich the discriminant features of few-shot images. The proposed Similarity Cache is derived from few-shot training images and public ImageNet [10] text labels, thus independent of the manually labeled class names. The design of the ImageNet-based Similarity Cache enables real-time on-site learning even in the absence of well-defined object names. By introducing inter-class similarity distribution as features, the proposed algorithm achieves superior classification accuracy compared with the state-of-the-art (SOTA) algorithms. Second, an image classification dataset "Temporary Objects on Construction Site" (TOCS) is created with 22 different object classes for few-shot on-site object classification. For evaluating the proposed algorithm, the object detection dataset SODA is modified by cropping the regions of interest for image classification. The study demonstrates the potential of few-shot learning with vision language models (VLM) to recognize temporary objects on construction sites. VLM-based few-shot learning is promising for recognizing construction objects utilizing Similarity Cache with open-source text labels, handcrafted text prompts and few-shot object detection.

The outline of the paper is arranged as follows. Section 2 reviews the computer vision algorithms used in construction, few-shot learning, and CLIP-based few-shot image classification. Section 3 introduces the proposed algorithm and compares it with the existing algorithms. Section 4 validates the proposed algorithm with extensive experiments on SODA and TOCS datasets. Section 5 discusses the limitations of the work and possible future work for improvement. Section 6 concludes the paper with a summary of contributions and implications.

## 2. Related work

### 2.1. Object detection and classification in construction

While AI-enabled computer vision holds promise for enhancing robot intelligence at a higher level, the development of construction robots equipped with intelligent vision capabilities is still in its early stages.

Existing research for on-site robots is focused on manipulation and programming within the individual construction task. On-site object detection and classification have been adopted mainly for project management [11]. In the last two decades, vision-based applications in construction have undergone rapid evolution, driven by the swift progress of computer vision algorithms [4]. A notable trend is that deep-learning-based computer vision algorithms have been dominant in various applications in construction.

Deep learning-based object detection algorithms can be classified as two categories: one-stage detection, such as You Only Look Once (YOLO), and two-stage detection, such as Faster R-CNN [12]. A Two-stage detector generates region proposals first and classifies each region into object categories. In contrast, a one-stage detector treats location as a regression problem and unifies the two targets (locations and categories) in a single framework.

Table 1 shows the algorithms, problem statement, training dataset, and object detection accuracy in construction. Both the object detection algorithms in Table 1 require large amounts of labeled data for training to improve detection accuracy. Capturing on-site images and videos on construction sites is a difficult task. Labeling objects in images and videos from construction sites is labor-intensive and time-consuming. Due to the variety of temporary objects used in construction, it is not feasible to have a comprehensive image dataset for every object. Limited training images will impact the performance of typical deep learning-based computer vision algorithms. Few-shot learning is desirable when limited training images are available.

### 2.2. Few-shot learning

Although conventional machine learning algorithms typically necessitate a substantial training dataset, humans exhibit the ability to learn and recognize new objects with just a few examples. To address this gap, the research community has explored various few-shot learning (FSL) algorithms to learn knowledge of novel objects in a data-efficient way. FSL has been applied in computer vision tasks, including few-shot image classification (FSIC), few-shot object detection (FSOD), and few-shot semantic segmentation [17]. Few-shot image classification aims to achieve human-level object recognition ability by giving only a few number of training image samples [18]. FSIC has been well explored by different types of algorithms based on transfer learning [19–21], meta-learning [22–25], and metric-learning [18,26]. Transfer learning algorithms operate on the assumption that knowledge obtained from data in the source domain can be transferred to the target domain through model fine-tuning. Meta-learning algorithms seek to learn generalizable knowledge by training the model on several different classification tasks. Metric learning algorithms focus on creating feature embeddings

**Table 1**  
Application of object detection algorithms in construction.

| Algorithms   | Problem statement                                                | Training dataset                                      | Detection accuracy | Author                  |
|--------------|------------------------------------------------------------------|-------------------------------------------------------|--------------------|-------------------------|
| Faster R-CNN | Detection of workers and equipment in real-time                  | 3029 images for workers and 498 images for excavators | 88.00%             | Fang et al., 2018 [13]  |
|              | Recognition of construction activities based on object detection | 7790 images containing 22 categories                  | 78.20%             | Luo et al., 2018 [14]   |
|              | Productivity analysis of crane transportation                    | 2467 images for buckets                               | 93.19%             | Wang et al., 2022 [15]  |
| YOLO         | Detect personal protective equipment in real time                | 14,500 images containing 6 categories                 | 76.86%             | Kumar et al., 2022 [16] |

and effective distance measurement. FSIC algorithms have been validated on public computer vision datasets. However, a research gap exists for few-shot image classification on challenging construction sites. Construction sites are highly dynamic and privacy-sensitive, making it difficult to collect large datasets of labeled images. A few recent studies have applied few-shot learning techniques to construction site data. For example, Cui et al. [27] proposed utilizing few-shot classification and contrast learning to recognize façade defects. Xu et al. [28] developed a meta-learning-based few-shot classifier for identifying structural damages from images. Kim et al. [29] applied few-shot object detection for learning and monitoring novel objects such as excavators on construction sites.

### 2.3. CLIP-based few-shot learning

Contrastive Language–Image Pre-training (CLIP) [9] is a milestone study that unifies vision and language in a consistent feature space. The critical insight of CLIP is to leverage natural language as a flexible prediction space to enable generalization and transfer. CLIP has achieved impressive zero-shot image classification performance on some of the public datasets. Inspired by CLIP, a new paradigm for few-shot image classification has been established by incorporating additional domain knowledge into CLIP. Tip-Adapter [8] is a representative work that combines CLP zero-shot prediction results with a visual cache. In the learning stage, Tip-Adapter proposed to use CLIP Image Encoder to extract visual embeddings from few-shot images and save them as “keys” in a Cache model. In the prediction stage, the test image is encoded with the same Image Encoder. The prediction label is obtained based on “values” of cache features that share the greatest affinity. The Tip-X [30] algorithm has introduced image-text similarity between images and the target dataset to provide better few-shot prediction performance. APE

[31] focuses on selecting significant feature channels via a prior refinement module. The selected feature channels have smaller inter-class similarity and higher inter-class disparity.

### 3. Few-shot CLIP adaptation with multimodal prototype

The few-shot image classification task is formulated as follows: Given Support Set  $\mathcal{S}_{support}$  with only a few support images with labels for each object class, the query images from Query Set  $\mathcal{S}_{query}$  need to be classified with pre-defined text labels. The notations mentioned in the paper are summarized in Appendix. Inspired by Tip-Adapter and TIP-X, the paper proposes a training-free CLIP-based few-shot learning algorithm in Fig. 1. Unlike traditional few-shot classification, the proposed algorithm does not require meta-training with base classes. Instead, all object classes within Target Dataset are regarded as novel classes. CLIP-based few-shot learning builds a training-free cache of support prototypes. Prediction is based on measuring the similarity between prototypes and query images in the embedding space. Two crucial questions need to be answered for CLIP-based few-shot learning. (1) How can we build a cache with representative prototypes for each class? (2) How can we effectively measure the similarity between the query image and prototypes? Addressing these questions necessitates considering different similarities. Table 2 summarizes the definition of different similarities: inter-image similarity, image-text similarity, and distribution similarity. Distribution similarity is the affinity between two distributions. Each distribution is a series of Image-text similarities between one image and multiple text prompts.

Fig. 1 shows the proposed algorithm with two phases. The training-free caching phase builds Visual Cache and the Similarity Cache by utilizing support images from  $\mathcal{S}_{support}$ . Subsequently, the prediction phase predicts the label of the query image from the  $\mathcal{S}_{query}$ . The overall

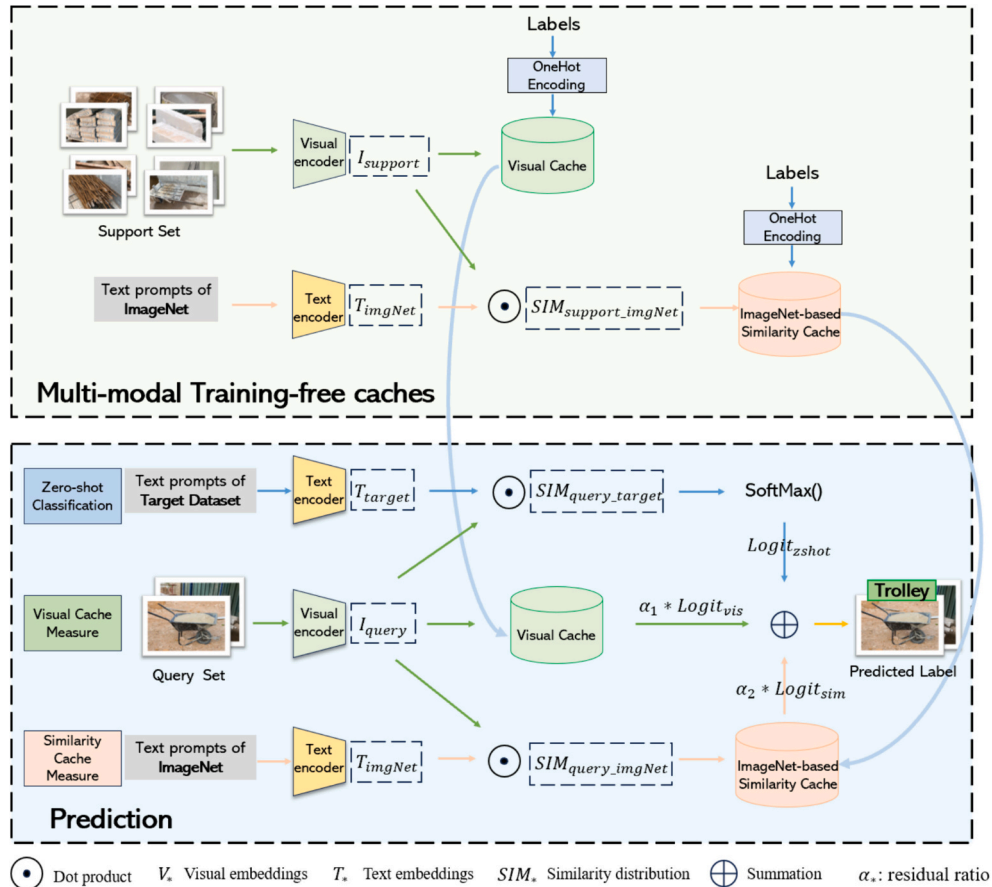


Fig. 1. Few-shot prediction with Visual Cache and Similarity Cache.

**Table 2**  
Different similarities.

| Types of Similarity            | Definition                                                                  |
|--------------------------------|-----------------------------------------------------------------------------|
| <b>Inter-image similarity</b>  | The similarity between two images in CLIP's encoding space                  |
| <b>Image-text similarity</b>   | The similarity between an image and a text prompt in CLIP's encoding space. |
| <b>Distribution similarity</b> | The similarity between two Image-text similarity distribution.              |

prediction results are composed of three branches: Zero-shot Classification, Visual Cache Measure, and Similarity Cache Measure.

Section 3.1 introduces the principles of CLIP zero-shot prediction. Section 3.2 introduces Visual Cache to adapt CLIP for temporary objects recognition with few-shot images. Section 3.3 introduces the proposed ImageNet-based Similarity Cache. Section 3.4 compares the proposed algorithm with the existing algorithms.

### 3.1. Zero-shot prediction from CLIP

Training contrastively on more than 400 million image-text pairs from the internet, CLIP [9] has learned powerful encoders for multi-modal representations. Fig. 2 (1) demonstrates the contrastive pre-training for maximizing the cosine similarity between matched image-text pairs. With the trained Image Encoder and Text Encoder, CLIP encodes images and text into an aligned embedding space. In Fig. 2 (2), a template “A photo of a [object]” is used to formulate text prompts by replacing “[object]” with text labels like “plane” and “car”. Text prompts are encoded as text embeddings  $T_1, T_2, \dots, T_N$  for  $N$  categories using CLIP Text Encoder. In Fig. 2 (3), the test image is encoded by CLIP Image Encoder as  $I_1$ . The cosine similarity between  $I_1$  and  $T_1, T_2, \dots, T_N$  is calculated to measure the affinity between the test image and  $N$  object labels. The test image can be classified as the text label that shares the largest similarity.

With the off-the-shelf CLIP, Fig. 3 shows the similarity matrix between 8 images of temporary objects along the horizontal axis and 8 text prompts along the vertical axis. The values indicate the cosine similarity of different image-text pairs in the embedding space. The values on the diagonal line are the largest, indicating correct matching of 8 image-text pairs. When two images have similar visual features, their cosine similarity values with one same text prompt could be close. Notably, in the red boxes, the similarity between the text prompt “This is a photo of a trowel” and the image “trowel” is 0.33, close to the similarity with image “shovel” as 0.31.

CLIP Image Encoder and CLIP Text Encoder are directly adopted as

“Visual Encoder” and “Text Encoder” in Fig. 1, with frozen neural network weights. Given a query image, the visual embedding  $I_{query} \in \mathbb{R}^{1 \times C}$  can be derived with CLIP Image Encoder. With text prompts from  $N$  labels in target dataset, text embedding  $T_{target} \in \mathbb{R}^{N \times C}$  is derived using CLIP Text Encoder.

In the first prediction branch of Fig. 1, “Zero-shot Classification” yields the predicted logits for  $N$  object classes as  $Logit_{zshot} \in \mathbb{R}^{1 \times N}$ . Cosine similarity between query visual features and  $N$  text prompts from the target dataset is measured as  $SIM_{query\_target} \in \mathbb{R}^{1 \times N}$  in Eq. (1).

$$SIM_{query\_target} = \text{norm}(I_{query}) * \text{norm}(T_{target})^T \quad (1)$$

Where  $\text{norm}(I_{query})$  stands for  $L2$  norm of each image features  $I_{query}$ .  $\text{norm}(T_{imgNet})^T$  stands for the transpose of  $L2$  norm of text features  $T_{target}$ . To convert cosine similarity  $SIM_{query\_target}$  to predicted probabilities of each class,  $Logit_{zshot} \in \mathbb{R}^{1 \times N}$  are calculated by applying *SoftMax* function in Eq. (2).  $Logit_{zshot}$  contains the predicted probability of each class from CLIP zero-shot.

$$Logit_{zshot} = \text{SoftMax}(SIM_{query\_target}) \quad (2)$$

### 3.2. Incorporate few-shot knowledge with visual cache

#### 3.2.1. Training-free caching with visual cache

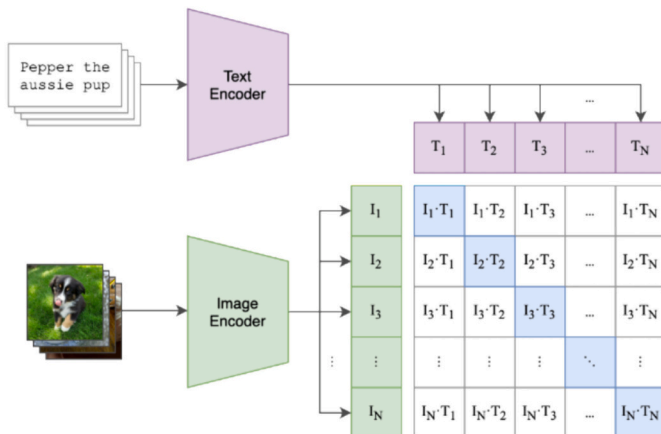
CLIP has shown outstanding zero-shot recognition ability on various public datasets. However, images of on-site temporary objects are usually kept private due to regulations and privacy. CLIP zero-shot performance on temporary objects is poor since it cannot access construction images. Following Tip-Adapter [8], Visual Cache is utilized in the study to incorporate new knowledge of the few-shot images with original knowledge from CLIP.

To build Visual Cache, image embeddings of few-shot images and their class labels are saved as “key-value” pairs. Let  $N$  be the number of classes in the target dataset and  $K$  be the number of images per class. The total number of images in the Support Set is  $n = K \times N$ . As shown in Fig. 1,  $n$  support images are encoded by Visual Encoder as image embeddings  $I_{support} \in \mathbb{R}^{n \times C}$  in Eq. (3). The labels of support images are encoded as One-Hot vectors  $L_{support} \in \mathbb{R}^{n \times N}$  in Eq. (4). Visual Cache is created by caching  $I_{support}$  and  $L_{support}$  as “key-value” pairs [8]. Image features  $I_{support}$  serves as weights for the prediction using Visual Cache. And  $L_{support}$  provides the true labels for the caches image features.

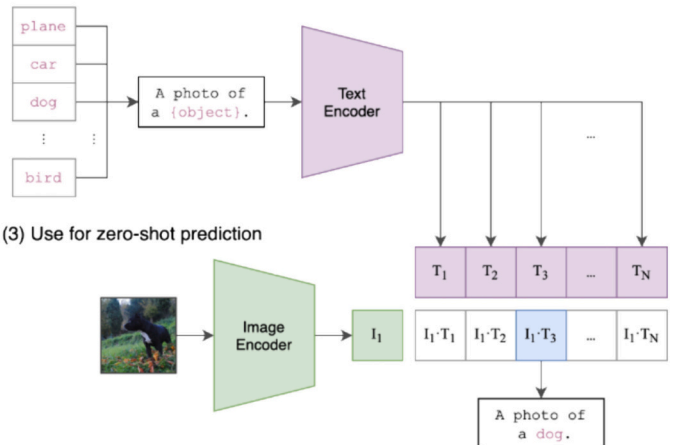
$$I_{support} = \text{VisualEncoder}(\mathcal{S}_{support}) \quad (3)$$

$$L_{support} = \text{OneHot}(\text{Labels}) \quad (4)$$

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

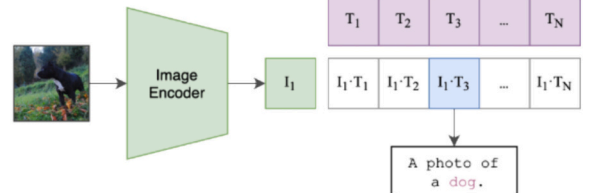


Fig. 2. CLIP and zero-shot prediction [9].



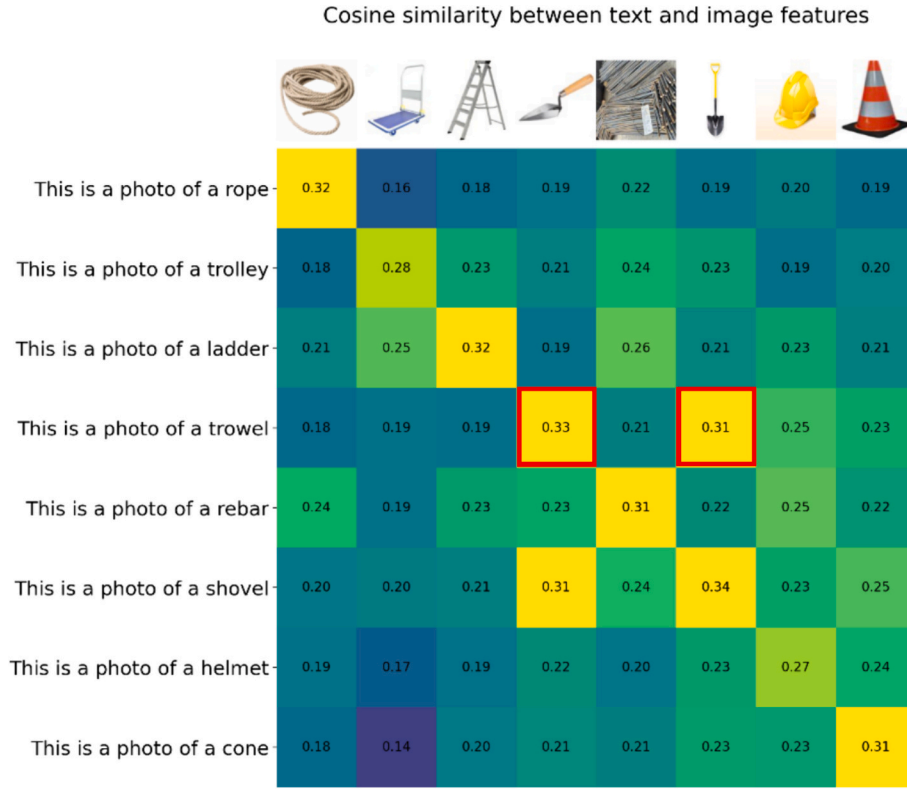


Fig. 3. CLIP zero-shot prediction on construction objects.

### 3.2.2. Prediction with visual cache

Training-free Visual Cache is utilized for the second prediction branch in Fig. 1: “Visual Cache Measure”. With the same Visual Encoder, the query image is encoded as  $I_{query}$ . Following Tip-Adapter [8], the affinity between  $I_{query}$  and  $I_{support}$  is calculated as  $A_{vis} \in \mathbb{R}^{1 \times n}$  in Eq. (5). The term  $1 - \text{norm}(I_{query}) * \text{norm}(I_{support})^T$  is equivalent to Euclidean distance between the query feature  $I_{query}$  and few-shot images’ feature in  $I_{support}$ . The exponential function converts query-key Euclidean distance to non-negative affinity  $A_{vis}$ . Hyperparameter  $\beta$  controls the sharpness of the function.

$$A_{vis} = \exp(-\beta[1 - \text{norm}(I_{query}) * \text{norm}(I_{support})^T]) \quad (5)$$

Multiplying the affinity  $A_{vis}$  with One-Hot encoding vector  $L_{support} \in \mathbb{R}^{1 \times N}$ , the confidence scores for each class are derived as  $\text{Logit}_{vis}' \in \mathbb{R}^{1 \times N}$  in Eq. (6).

$$\text{Logit}_{vis}' = A_{vis} L_{support} \quad (6)$$

### 3.3. Enhancing CLIP with ImageNet-based similarity cache

#### 3.3.1. Training-free caching with ImageNet-based similarity cache

CLIP undergoes contrastive training to maximize inter-modal cosine similarity, with less emphasis on prioritizing intra-modal similarity. Consequently, the similarity distances among intra-modal image features used in Tip-Adapter might not accurately adhere to the actual distribution. Considering this gap, TIP-X [30] introduces the image-text similarity cache with text labels from Target Dataset. In this study, the cache of image-text similarity is referred to as the Similarity Cache. The study identifies two limitations of TIP-X for recognition of temporary objects on construction sites. First, TIP-X necessitates text labels of Target Dataset. Naming object labels on construction sites consistently and accurately could be challenging without expert knowledge. Second, even if temporary objects are aptly named, CLIP may not have the

knowledge to relate the textual names with the on-site images.

Given the constraints of existing Tip-X, this paper proposes to employ abundant object labels from open source to create Similarity Cache. This approach allows the creation of text prompts beyond the boundaries of the Target Dataset. Specifically, this study creates ImageNet-based Similarity Cache with ImageNet object labels. Given any image-text pair, CLIP can generate a cosine similarity score. Fig. 4 shows CLIP zero-shot classification scores of temporary objects with the five most related text labels from CIFAR-100 dataset [32]. The “cone” exhibits the highest similarity score with “rocket”, while the rope shares similarities with a “snake”. The similarity distribution over different text labels is discriminant for each object class. The distinctive inter-class similarity distribution motivates this study to establish the Similarity Cache with open-source text labels.

Specifically, text labels for building Similarity Cache are from open-source public datasets. In the study,  $M$  text prompts are created from ImageNet class labels. The basic template “This is a photo of X” is used for text prompts, where X denotes the object name.  $T_{imgNet} \in \mathbb{R}^{M \times C}$  represents the text features of ImageNet text prompts encoded by CLIP Text Encoder. By utilizing inter-class similarity distribution as features, this study calculates cosine similarity for support image features  $I_{support}$  with ImageNet text features  $T_{imgNet}$ , denoted as  $SIM_{support\_imgNet} \in \mathbb{R}^{n \times M}$ . Mathematically, this is expressed as Eq. (7).

$$SIM_{support\_imgNet} = \text{norm}(I_{support}) * \text{norm}(T_{imgNet})^T \quad (7)$$

$SIM_{support\_imgNet}$  is saved as “keys” for Similarity Cache, alongside the One-Hot encoding  $L_{support}$  as corresponding “values”. In this way, each object class derives a similarity distribution over ImageNet text labels. This distribution is unique for each class and could be utilized for characterizing the specific object.

#### 3.3.2. Prediction with ImageNet-based similarity cache

For the third prediction branch “Similarity Cache Measure” in Fig. 1, prediction  $\text{Logit}_{sim}$  from ImageNet-based Similarity Cache is derived.

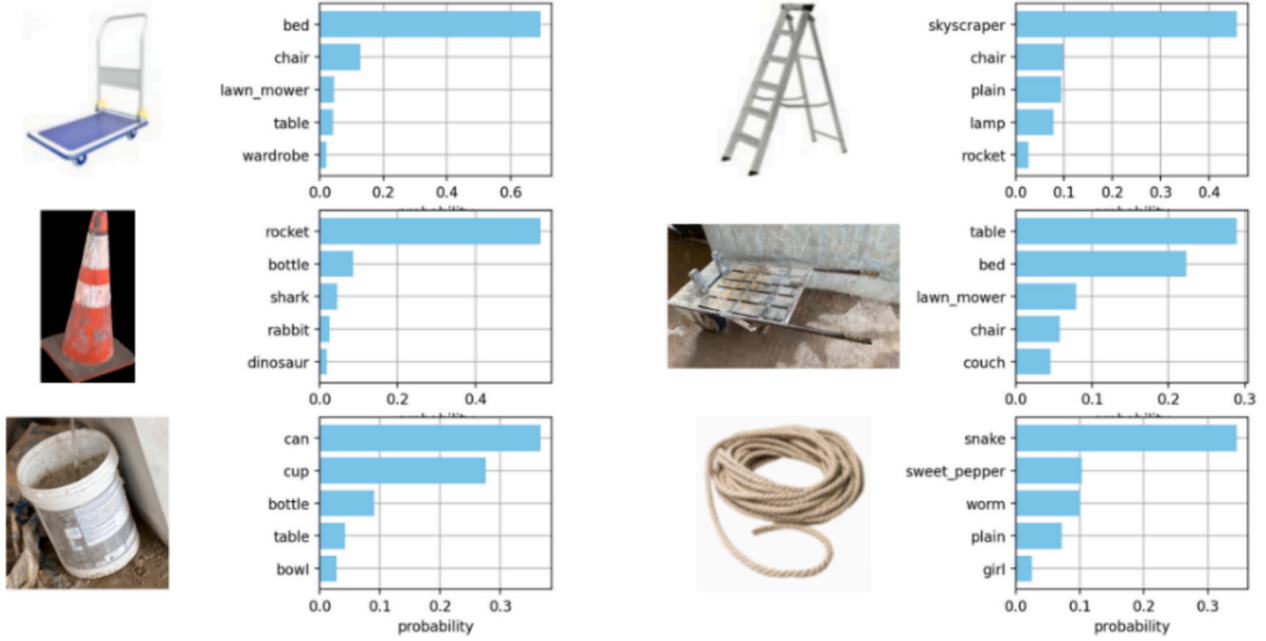


Fig. 4. CLIP zero-shot classification scores with cross-dataset labels.

Given the query image feature  $I_{query}$ , cosine similarity distribution between  $I_{query}$  and  $T_{imgNet}$  is calculated as  $SIM_{query\_imgNet} \in \mathbb{R}^{1 \times M}$  in Eq. (8).

$$SIM_{query\_imgNet} = \text{norm}(I_{query}) * \text{norm}(T_{imgNet})^T \quad (8)$$

The measure of affinity between query similarity distribution  $SIM_{query\_imgNet}$  and support similarity distribution  $SIM_{support\_imgNet}$  is critical. Instead of using Kullback-Leibler (KL) divergence as TIP-X, this study proposes to measure the affinity between similarity distributions with Eq. (9). Min-Max normalization is applied to the cosine similarity of  $SIM_{query\_imgNet}$  and  $SIM_{support\_imgNet}$ . The purpose of Min-Max normalization is to ensure that all features have the same scale and to bring the values of the features into a certain range. With the affinity of query distribution to the support distributions in Similarity Cache, prediction of  $Logit_{sim}$  in Eq. (10) is the matrix production of  $A_{sim}$  and One-Hot of support labels  $L_{support}$ .

$$A_{sim} = \text{MinMax} \left[ \text{norm}(SIM_{query\_imgNet}) * \text{norm}(SIM_{support\_imgNet})^T \right] \quad (9)$$

$$Logit_{sim} = A_{sim} L_{support} \quad (10)$$

### 3.4. Formulation of unified prediction

The unified prediction result is the weighted sum of CLIP zero-shot prediction, Visual Cache prediction, and Similarity Cache prediction. In TIP-X,  $A_{sim}$  is rescaled to the same range of  $A_{vis}$ . However, the range of  $Logit_{vis}$  and  $Logit_{sim}$  will change with the different number of learning shots. Rather than following TIP-X to rescale  $A_{sim}$ , this study rescales  $Logit_{vis}$  and  $Logit_{sim}$  to the same range of  $Logit_{zshot}$  in Eq. (11) and (12), denoted as  $Logit_{vis} \in \mathbb{R}^{1 \times N}$  and  $Logit_{sim} \in \mathbb{R}^{1 \times N}$ . The rescale function in Eq. (13) normalizes a vector X is to the range of another vector Y. The term  $\frac{(X - X.min)}{(X.max - X.min)}$  normalizes X to the range [0,1]. Multiplying the normalized X with the range of Y and shifting the data to  $Y.min$  derives the aligned X and Y.

$$Logit_{vis} = \text{rescale}(Logit_{vis}, Logit_{zshot}) \quad (11)$$

$$Logit_{sim} = \text{rescale}(Logit_{sim}, Logit_{zshot}) \quad (12)$$

$$F(X, Y) = \frac{(X - X.min)}{(X.max - X.min)} * (Y.max - Y.min) + Y.min \quad (13)$$

The combined prediction from three prediction branches  $Logit \in \mathbb{R}^{1 \times N}$  is calculated in Eq. (14). Hyperparameters  $\alpha_1$  and  $\alpha_2$  are the weights of the learned few-shot knowledge from Visual Cache and ImageNet-based Similarity Cache. The predicted label is selected based on the  $N$  probability scores for object classes within  $Logit$ .

$$Logit = Logit_{zshot} + \alpha_1 * Logit_{vis} + \alpha_2 * Logit_{sim} \quad (14)$$

### 3.5. Comparison with existing algorithms

This section compares the proposed algorithm with CLIP zero-shot, Tip-Adapter, and TIP-X. The features of different algorithms are shown in Table 4. Tip-Adapter introduces Visual Cache and combines the prediction of Visual Cache with CLIP zero-shot prediction. TIP-X introduces Similarity Cache with labels of Target Dataset. In contrast, the proposed algorithm proposes ImageNet-based Similarity Cache. Different caches are involved in the compared algorithms. Table 3 summarizes the definition of different Caches.

Compared with TIP-X, the proposed algorithm has three critical differences. The first difference is utilizing open-source text labels (ImageNet) to create Similarity Cache. Noticing CLIP's ability to

Table 3

Comparison of caches.

| Types of Caches             | Source of Text Prompts          | Require training images? | Require label of training images? | Data in the cache                                   |
|-----------------------------|---------------------------------|--------------------------|-----------------------------------|-----------------------------------------------------|
| Visual Cache                | N.A.                            | Yes                      | Yes                               | Image embeddings                                    |
| Similarity Cache (TIP-X)    | Target Dataset                  | Yes                      | Yes                               | Image-text similarity distribution (Target Dataset) |
| Similarity Cache (Proposed) | Public Datasets (e.g. ImageNet) | Yes                      | No                                | Image-text similarity distribution (ImageNet)       |

**Table 4**  
Comparison of different algorithms.

| Algorithms Item                            | CLIP Zero-shot    | Tip-Adapter       | TIP-X                       | The Proposed                                |
|--------------------------------------------|-------------------|-------------------|-----------------------------|---------------------------------------------|
| With few-shot knowledge?                   | —                 | +                 | +                           | +                                           |
| With Visual Cache?                         | —                 | +                 | +                           | +                                           |
| With Similarity Cache from Target Dataset? | —                 | —                 | +                           | —                                           |
| With Similarity Cache from ImageNet?       | —                 | —                 | —                           | +                                           |
| Measure of inter-image similarity          | N.A.              | Beta-function     | Beta-function               | <b>Beta-function</b>                        |
| Measure of image-text similarity           | cosine similarity | cosine similarity | cosine similarity           | <b>cosine similarity</b>                    |
| Measure of distribution similarity         | N.A.              | N.A.              | KL-divergence               | <b>Normalized cosine similarity Eq. (9)</b> |
| Hyperparameters                            | N.A.              | $\beta, \alpha_1$ | $\beta, \alpha_1, \alpha_2$ | $\beta, \alpha_1, \alpha_2$                 |

**Table 5**  
Number of images per class for modified SODA.

| Hook   | Fence    | Board  | Vest         | Slogan        |
|--------|----------|--------|--------------|---------------|
| 672    | 2586     | 2252   | 350          | 1160          |
| Hopper | Scaffold | Cutter | Handcart     | Rebar         |
| 537    | 5240     | 251    | 414          | 2233          |
| Helmet | Brick    | Wood   | Electric box | <b>SUM</b>    |
| 42     | 825      | 2824   | 451          | <b>19,837</b> |

**Table 6**  
Number of images in TOCS.

| Bucket                | Cardboard box | Concrete bar | Concrete block | Concrete vibrator | Cone        |
|-----------------------|---------------|--------------|----------------|-------------------|-------------|
| 8                     | 3             | 3            | 4              | 5                 | 5           |
| Electric scissor lift | Hard hat      | Ladder       | Lighting tower | Metal bar         | Mould block |
| 3                     | 5             | 6            | 2              | 3                 | 4           |
| PVC pipe              | Rebar coil    | Rebar        | Sandbag        | Shelf             | Steel bar   |
| 3                     | 4             | 6            | 6              | 3                 | 2           |
| Trolley               | Water tank    | Wood bar     | Wood frame     | <b>SUM</b>        |             |
| 5                     | 3             | 4            | 3              | <b>90</b>         |             |

**Table 7**  
Dataset split.

|                     | SODA                                 | TOCS                   |
|---------------------|--------------------------------------|------------------------|
| <b>Training Set</b> | 1, 2, 4, 8, 16 images for each class | 1 image for each class |
| <b>Valid Set</b>    | 2915 images                          | 20 images              |
| <b>Test Set</b>     | 3037 images                          | 48 images              |

**Table 8**  
Number of text prompts.

|                                  | SODA                                                 | TOCS | ImageNet |
|----------------------------------|------------------------------------------------------|------|----------|
| <b>Template for text prompts</b> | "This is a photo of X" (replace "X" with class name) |      |          |
| <b>Number of classes</b>         | 14                                                   | 22   | 1000     |

measure similarity between any given image-text pairs, this study proposes utilizing open-source text prompts to build even stronger Similarity Cache. The work specifically explores building the Similarity Cache with text labels of publicly available ImageNet [10].

**Table 9**  
Classification accuracy on SODA.

| Few-shot Setup                | 1            | 2            | 4            | 8            | 16           |
|-------------------------------|--------------|--------------|--------------|--------------|--------------|
| CLIP zero-shot: 23.17         |              |              |              |              |              |
| Tip-Adapter                   | 24.61        | 28.01        | 34.97        | 40.79        | 53.77        |
| TIP-X                         | 43.21        | 55.04        | 60.39        | 60.66        | 69.33        |
| <b>The Proposed Algorithm</b> | <b>48.42</b> | <b>56.29</b> | <b>62.41</b> | <b>63.85</b> | <b>73.07</b> |

**Table 10**  
Classification accuracy on TOCS.

| Random Dataset Splits         | Split 1      | Split 2      | Split 3      | Split 4      | Split 5      | Average      |
|-------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|
| CLIP zero-shot                | 60.42        | 58.33        | 56.25        | 58.33        | 58.33        | 58.33        |
| Tip-Adapter                   | 62.50        | 60.42        | 58.33        | 58.33        | 60.42        | 60.00        |
| TIP-X                         | <b>83.33</b> | 81.25        | 75.0         | 75.0         | 72.92        | 77.50        |
| <b>The Proposed Algorithm</b> | <b>83.33</b> | <b>85.42</b> | <b>77.08</b> | <b>81.25</b> | <b>89.58</b> | <b>83.33</b> |

**Table 11**  
Computation time for different algorithms.

| Algorithms                    | Number of search steps for hyper parameter | Computation Time (seconds) on TOCS | Computation Time (seconds) on SODA |
|-------------------------------|--------------------------------------------|------------------------------------|------------------------------------|
| CLIP zero-shot                | 0                                          | 8.90                               | 9.10                               |
| Tip-Adapter                   | 40,00                                      | 31.80                              | 19.17                              |
| TIP-X                         | 200,000                                    | 126.67                             | 127.42                             |
| <b>The Proposed Algorithm</b> | <b>40,000</b>                              | <b>75.10</b>                       | <b>56.52</b>                       |

**Table 12**  
Classification accuracy with different matching functions.

| Accuracy on SODA        |              |              |              |              |              |              |
|-------------------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Few-shot Setup          | 1            | 2            | 4            | 8            | 16           | Average      |
| KL-divergence (TIP-X)   | 43.21        | 55.04        | <b>60.39</b> | 60.66        | 69.33        | 57.73        |
| <b>Proposed Eq. (9)</b> | <b>44.49</b> | <b>55.08</b> | 59.68        | <b>62.31</b> | <b>70.65</b> | <b>58.44</b> |
| Accuracy on TOCS        |              |              |              |              |              |              |
| Random Dataset Splits   | Split 1      | Split 2      | Split 3      | Split 4      | Split 5      | Average      |
| KL divergence (TIP-X)   | <b>83.33</b> | 81.25        | 75.0         | 75.0         | 72.92        | 77.50        |
| <b>Proposed Eq. (9)</b> | 81.25        | <b>87.5</b>  | <b>79.17</b> | <b>83.33</b> | <b>85.42</b> | <b>83.33</b> |

**Table 13**  
Optimal hyperparameters with different learning shots.

| Few-shot Setup        |              | 1                              | 2     | 4     | 8     | 16     |
|-----------------------|--------------|--------------------------------|-------|-------|-------|--------|
| Test Accuracy on SODA |              | 48.42                          | 56.29 | 62.41 | 63.85 | 73.07  |
| Hyperparameters       | Search scale | Optimal values in search scale |       |       |       |        |
| $\beta$               | [0, 50]      | 28.04                          | 28.04 | 27.50 | 33.53 | 36.00  |
| $\alpha_1$            | [0,200]      | 9.50                           | 9.50  | 12.57 | 60.00 | 170.00 |
| $\alpha_2$            | [0,20]       | 3.07                           | 4.06  | 2.6   | 1.09  | 2.09   |

The second difference is the measure of distribution similarity in Similarity Cache. This study proposes Min-Max normalized cosine similarity in Eq. (9) to measure the affinities of two distributions in Similarity Cache. This contrasts with TIP-X, which employed KL-divergence for measuring distributional affinity. While KL-divergence excels at comparing probability distributions, the cosine similarity do not strictly adhere to such distributions. Instead, they represent geometric similarities between vectors. Furthermore, unlike KL-divergence which lacks

**Table 14**  
Handcrafted text prompts.

| Object labels | Text prompts                                                                                                                                        |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Bucket        | A bucket is usually round container for catching, holding, or carrying liquids or solids.                                                           |
| Hard hat      | A hard hat made of a strong material which is worn to protect the head especially of someone who builds buildings.                                  |
| Ladder        | A ladder is a vertical or inclined set of rungs or steps used for climbing or descending.                                                           |
| Rebar         | Rebar (short for reinforcing bar), known when massed as reinforcing steel or reinforcement steel.                                                   |
| Rope          | A rope is a group of yarns, plies, fibers, or strands that are twisted or braided together into a larger and stronger form.                         |
| Traffic cone  | Traffic cones, also called pylons, witches' hats, road cones, highway cones, safety cones, channelizing devices, construction cones, or just cones. |
| Trolley       | Construction trolleys are used to transport and carry heavy loads.                                                                                  |

symmetry, cosine similarity exhibits this desirable property. Therefore, this study proposes a Min-Max normalized cosine similarity measure (Eq. 9) to measure the affinity between query and support distributions.

The third difference is the mechanism for aggregating prediction logits. Eq. (15) details the approach used by TIP-X, while the proposed algorithm adopts a different strategy, as shown in Eq. (16). TIP-X rescales  $A_{sim}$  to the range of  $A_{vis}$  as in Eq. (15). This study proposes to rescale  $A_{vis}L_{support}$  and  $A_{sim}L_{support}$  to the range of  $Logit_{zshot}$  in Eq. (16). Let  $A_j$  denote the  $j$ th column of  $A_{sim}$ , the rescaled  $A_j \in [0, 1]$ . Let  $C_j$  denote the  $j$ th column of  $Logit_{sim}'$ .  $L_{ij} \in [0, 1]$  denotes the element of the  $i$ th row and the  $j$ th column in  $L_{support}$ . It can be derived that  $C_j = \sum_{i=1}^n A_j L_{ij}$  will be in the range of  $[0, n]$ . Therefore, the range of  $Logit_{sim}' = A_{sim}L_{support}$  depends on  $n$ , the number of support images. Similarly, the range of  $Logit_{vis}' = A_{vis}L_{support}$  is changing with the  $n$ . The changing range of  $Logit_{vis}'$  and  $Logit_{sim}'$  makes the ratio  $Logit_{zshot} : Logit_{vis}' : Logit_{sim}'$  inconsistent. Therefore, it is difficult to set proper  $\alpha_1$  (the weight of Visual Cache) and  $\alpha_2$  (the weight of Similarity Cache) in Eq. (15). In the proposed Eq. (16),  $Logit_{vis}'$  and  $Logit_{sim}'$  are rescaled into the range of CLIP zero-shot  $Logit_{zshot}$ . This modification makes the ratio between different prediction from three caches fixed as  $Logit_{zshot} : Logit_{vis}' : Logit_{sim}' = 1 : \alpha_1 : \alpha_2$ . By fixing the ratio between weights of three prediction logits, the hyperparameter search process would become more efficient (validated in section 4.3.3).

$$logit_{TIP-X} = Logit_{zshot} + \alpha_1 * A_{vis}L_{support} + \alpha_2 * rescale(A_{sim}, A_{vis})L_{support} \quad (15)$$

$$logit_{proposed} = Logit_{zshot} + \alpha_1 * rescale(A_{vis}L_{support}, Logit_{zshot}) + \alpha_2 * rescale(A_{sim}L_{support}, Logit_{zshot}) \quad (16)$$

#### 4. Few-shot classification of on-site temporary objects

To evaluate the performance of the proposed few-shot image classification algorithm for on-site objects, experiments are conducted on a public dataset Site Object Detection Dataset (SODA) [33], and the newly developed dataset named Temporary Objects on Construction Site (TOCS). Section 4.1 introduces the experiment setup and dataset details. Section 4.2 introduces the experiment details for building Visual Cache and Similarity Cache. Classification accuracy and computation efficiency compared to the existing algorithms are discussed in Section 4.3. To validate each component in the proposed algorithm, Section 4.4 discusses individual performance of different Caches (Section 4.4.1), the proposed Similarity Cache matching function (Section 4.4.2), hyperparameters (Section 4.4.3), and text prompts (Section 4.4.4).

#### 4.1. Experiment settings

##### 4.1.1. Pre-process of SODA dataset

While the publicly available SODA dataset is originally designed for object detection tasks, this study tailored it for image classification. The original SODA dataset contains 19,846 images with 286,201 object instances across 15 object classes (Fig. 6). To adapt it for classification, we implemented a customization process, as illustrated in Fig. 5.

This involved: 1) cropping Regions of Interest (ROIs) for each object using their annotated bounding boxes and class labels, and 2) applying specific filtering criteria. Considering the CLIP Image Encoder's input requirement of  $224 \times 224$  pixels, we excluded images with dimensions smaller than this threshold to ensure adequate resolution. Additionally, images containing "person" are removed to maintain focus on temporary site objects, aligning with the study's objectives. These steps yielded a customized SODA dataset for classification comprising 14 object classes and 19,837 images (Table 5). Fig. 7 visualizes the image dimensions within this dataset, revealing a minimum width and height of 224 pixels and mean values of 436 pixels and 575 pixels for height and width, respectively.

##### 4.1.2. TOCS dataset

This study introduces a new small-scale dataset for further evaluating the proposed algorithm's capabilities: 'Temporary Objects on Construction Site' (TOCS) dataset. Captured directly at construction sites, the TOCS dataset encompasses 90 images representing 22 distinct object classes, offering additional diversity and real-world complexity. Fig. 9 showcases example images from TOCS. Table 6 illustrates the class-wise image distribution, while Fig. 8 visualizes the overall image dimension distribution. Compared to SODA, the TOCS dataset offers a smaller scale (90 images) but with substantially greater class diversity (22 classes). This deliberate design choice allows for focused evaluation of the algorithm's adaptability to new classes and real-world construction scene variations, while minimizing resource requirements.

#### 4.2. Build training-free caches

To ensure rigorous evaluation of the proposed algorithm, each dataset is divided into three splits as shown in Table 7: Training Set, Validation set and Test Set. Training Set contains support images utilized to build Visual Cache and Similarity Cache. Validation Set is adopted for searching and optimizing hyperparameters  $\beta$ ,  $\alpha_1$  and  $\alpha_2$ . The prediction accuracy is evaluated using query images in Test Set. For the SODA dataset, the study evaluated few-shot classification accuracy across a comprehensive range of shots (1, 2, 4, 8, and 16), offering an understanding of performance under varying data constraints. In contrast, the TOCS dataset, due to its inherently limited sample size per class, is subjected to 1-shot classification evaluation to gauge its effectiveness in extreme low-data scenarios. To facilitate direct comparison with existing algorithms, the experiments adopted the Image Encoder and Text Encoder from the pretrained CLIP model with a ResNet-50 backbone. Training images underwent preprocessing procedures aligned with CLIP's requirements, including resizing to a fixed dimension of  $224 \times 224$  pixels using the PyTorch library function "RandomResizedCrop". Adhering to the data augmentation strategy employed in Tip-Adapter, each training image is augmented through random scaling and flipping with a predetermined probability.

The preprocessed training images are subsequently encoded into 1024-dimensional training features using the off-the-shelf CLIP Image Encoder. These features serve as "keys" within the Visual Cache, while the corresponding One-Hot encoded labels constitute the "values". In parallel, the Similarity Cache is constructed. Text prompts are generated from either the target datasets themselves (SODA and TOCS, adhering to the TIP-X framework) or the ImageNet dataset (proposed approach). Text labels for ImageNet-based Similarity Cache are derived from ImageNet. The prompt template "This is a photo of X" is employed,



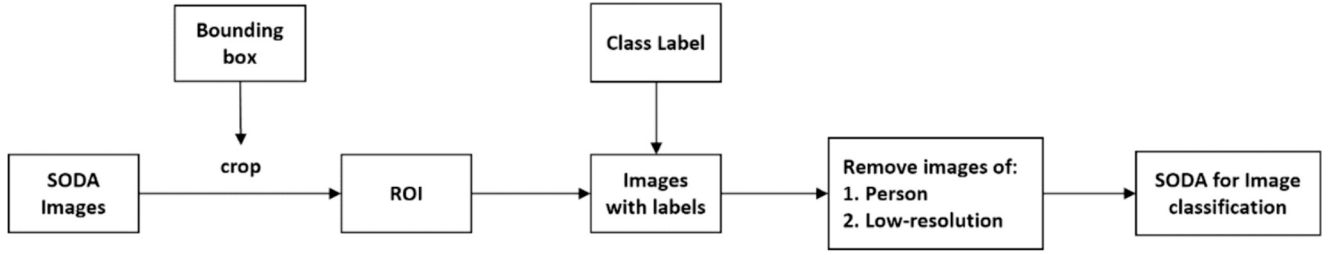


Fig. 5. Preprocessing of SODA images.

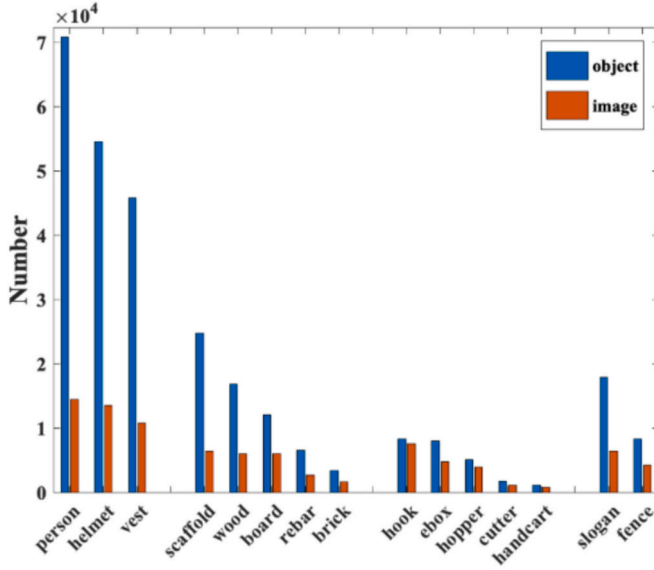


Fig. 6. Number of objects and images [33].

except for the hand-crafted prompts in Section 4.4.4. Table 8 summarizes the number of classes within each dataset. Text prompts are encoded into text embeddings using CLIP's Text Encoder. The Similarity Cache is formulated with cosine similarity scores between training image embeddings and text embeddings as "keys", and One-Hot encoded labels as "values".

#### 4.3. Few-shot classification performance

The images from Validation Set are leveraged to optimize hyperparameters  $\beta$ ,  $\alpha_1$  and  $\alpha_2$ . The hyperparameter  $\beta$  governs the sharpness of the exponential function within Eq. (5), while  $\alpha_1$  and  $\alpha_2$  determine the weights assigned to predictions from the Visual Cache and Similarity Cache during their combined prediction in Eq. (14). The search ranges for the hyperparameters are meticulously defined based on their observed impact on validation accuracy. The hyperparameter  $\beta$  is explored within a range of 0.1 to 50, with 100 search steps.  $\alpha_1$  is searched within a range of 0 to 200, adopting 20 steps, and  $\alpha_2$  is examined within a range of 0 to 20, utilizing 20 steps. The parameter searching process identifies the optimal hyperparameter configuration that yields the highest prediction accuracy on the Validation Set. The derived optimal hyperparameters are subsequently employed for prediction tasks on the Test Set. The proposed algorithm's performance is benchmarked against CLIP zero-shot, Tip-Adapter, and TIP-X, facilitating a comprehensive evaluation of its efficacy.

##### 4.3.1. Accuracy comparison on SODA

Fig. 10 and Table 9 present the few-shot classification accuracy of different algorithms on the SODA dataset. Among all the compared algorithms, the proposed algorithm shows the highest classification accuracy across all different number of learning shots.

CLIP zero-shot demonstrates a limited performance, achieving only 23.17% accuracy. This highlights the challenge CLIP faces when dealing with real-world temporary objects from diverse construction sites. Even with the same text labels, "Fence" and "Scaffolding" can manifest in vastly different shapes and colors across construction projects. Due to its limited exposure to images on construction sites, CLIP struggles to

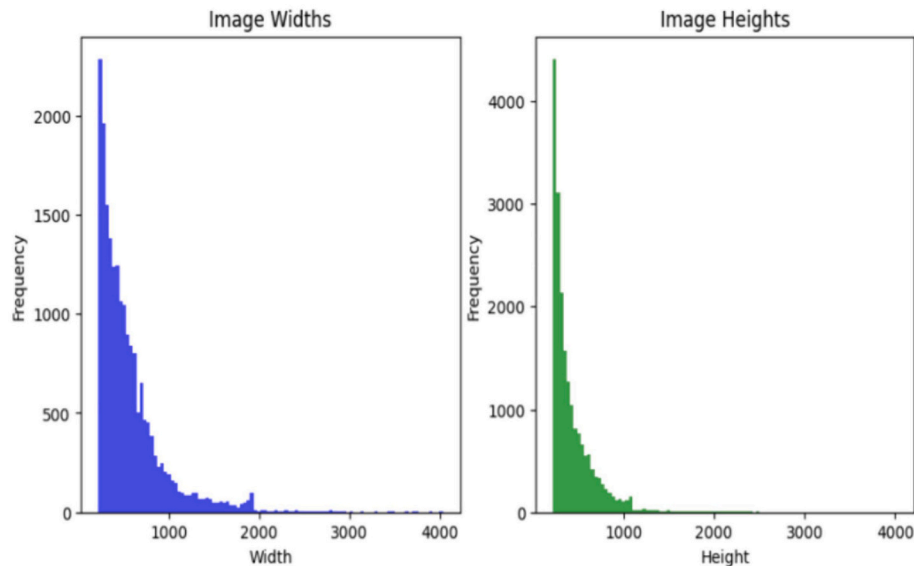


Fig. 7. Image dimensions of customized SODA.

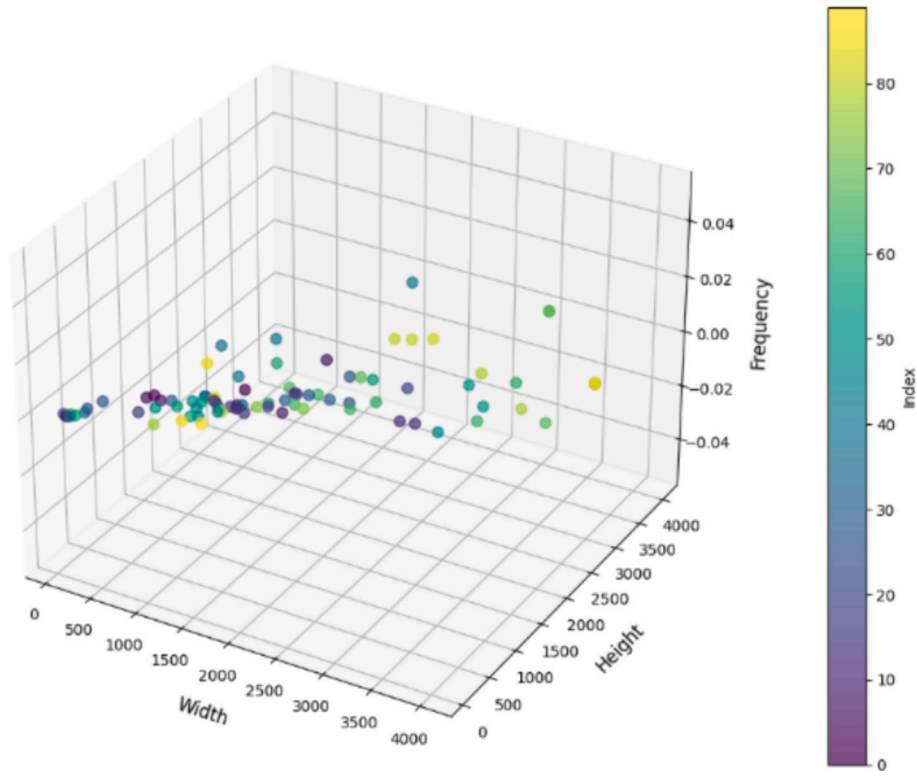


Fig. 8. Image dimensions in TOCS.



Fig. 9. Sample images in TOCS.

recognize these variations. While CLIP zero-shot only shows 23.17% accuracy, Tip-Adapter exhibits a linear increase in accuracy with more learning shots. At the 16-shot setup, it reaches 53.77%, suggesting the effectiveness of its Visual Cache in learning class-specific visual features. With increasing shots, Visual cache provides a broader range of potential

visual features for matching. TIP-X further propels few-shot performance compared to Tip-Adapter, achieving 43.21% accuracy with only one training shot, a substantial improvement over the Tip-Adapter's 24.61%. This consistent outperformance across 2, 4, 8, and 16 shots underscores the efficacy of TIP-X's Similarity Cache constructed with

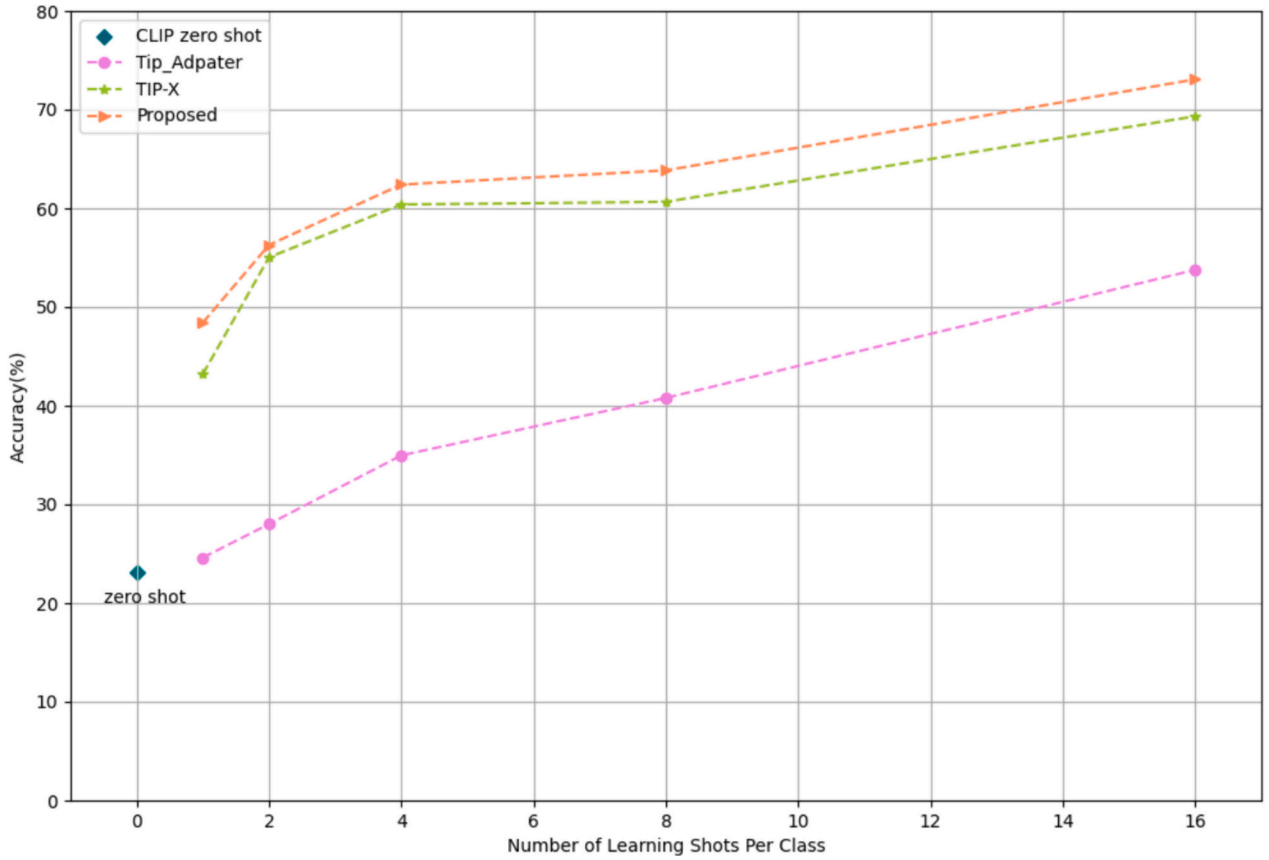


Fig. 10. Few-shot classification performance on SODA.

image-text similarity based on target text labels. Despite the notable improvement of both Tip-Adapter and TIP-X over CLIP zero-shot, the proposed algorithm surpasses all compared algorithms. With only one available learning shot, the proposed algorithm exhibits a 23.81% accuracy gain over Tip-Adapter and a 5.21% gain over TIP-X. The 5.21% gain translates to approximately 158 more accurately predicted images on the SODA Test Set.

#### 4.3.2. Accuracy comparison on TOCS

The few-shot classification accuracy over TOCS dataset is shown in Fig. 11 and Table 10. Consistent with performance on SODA, the proposed algorithm demonstrates the top classification accuracy across all dataset splits.

Since the proposed TOCS dataset is relatively small, 1-shot performance is evaluated on 5 randomly generated Train-Valid-Test splits. Whereas CLIP zero-shot achieved only 23.17% accuracy on SODA, it exhibits a significant jump to an average of near 58% on TOCS. This improvement can be attributed to two factors. Firstly, textual descriptions within TOCS are more nuanced, such as specifying the “Rebar coil” shape precisely. Secondly, objects like “Bucket” and “Hard hat” in TOCS are likely encountered by CLIP during its training, facilitating easier recognition. With only one training image per class in Visual Cache, Tip-Adapter exhibits a modest 2% accuracy gain compared to CLIP zero-shot. In contrast, leveraging the Similarity Cache with TOCS’s labels, TIP-X achieves a substantial average accuracy of 78%. Ultimately, the proposed algorithm, empowered by both Visual Cache and ImageNet-based Similarity Cache, surpasses all compared algorithms with an average accuracy of 83%. Notably, in Split 5, the proposed algorithm reaches a remarkable 89.58% accuracy, representing a 17% improvement over existing TIP-X.

The consistent performance on SODA and TOCS reaffirms the effectiveness of the proposed algorithm. By introducing ImageNet-based

Similarity Cache, the inter-class similarity distribution with open-source text labels substantially improves classification accuracy. Similarity Cache has shown stronger representation ability using text labels from ImageNet than Target Dataset. There are 1000 text labels in ImageNet while only 14 text labels in modified SODA and 22 text labels in TOCS. More comprehensive inter-class similarity information is captured in ImageNet-based Similarity Cache.

#### 4.3.3. Computation efficiency comparison

This section compares the efficiency of the proposed algorithms with CLIP zero-shot, Tip-Adapter, and TIP-X. Despite achieving the highest classification accuracy, the proposed algorithm requires significantly less computation time than TIP-X. This efficiency stems from TIP-X’s requirement for 200,000 optimization steps for searching hyperparameters ( $\beta$ ,  $\alpha_1$ ,  $\alpha_2$ ). In contrast, the proposed algorithm leverages rescaled  $Logit_{vis}$  and  $Logit_{sim}$ , requiring only 40,000 search steps. The computation time for each algorithm is shown in Table 11. All algorithms are executed with a 1-shot setup on the same desktop equipped with an Nvidia RTX 2080 GPU. CLIP zero-shot exhibits the fastest execution time on both datasets (under 10 s) due to its lack of hyperparameter optimization. TIP-X and the proposed algorithm executes for a longer time than Tip-Adapter for optimizing weight of Similarity Cache  $\alpha_2$  on Validation Set.

#### 4.4. Ablation study

##### 4.4.1. Comparison on different caches

Eq. (14) defines the overall prediction as a weighted sum of individual contributions from CLIP zero-shot, Visual Cache, and Similarity Cache. To delve into the role of each component, this section analyzes their independent performance. In Fig. 12 (a), “Visual” represents the Visual Cache, “SODA sim” denotes the Similarity Cache with SODA’s

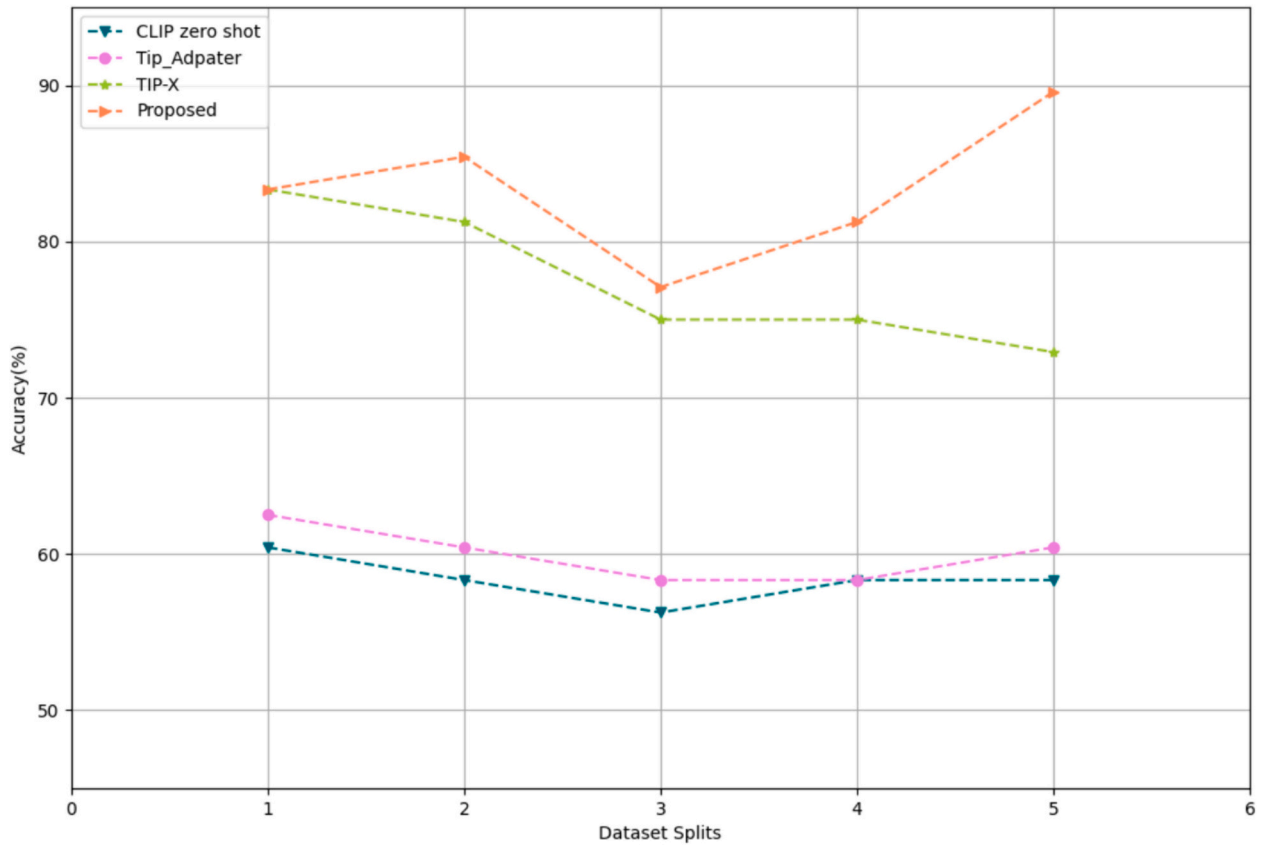


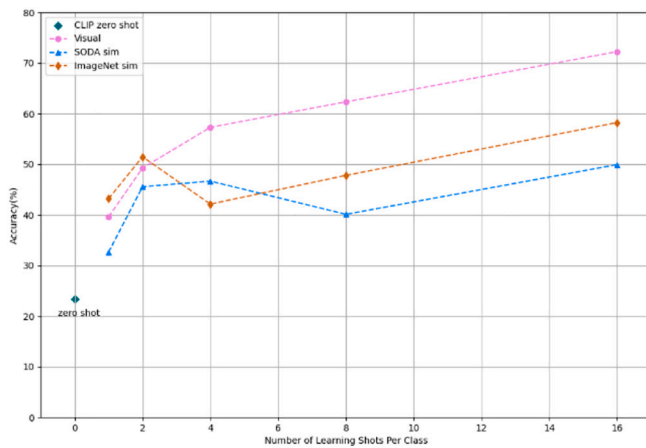
Fig. 11. Few-shot classification performance on TOCS.

text labels, and “ImageNet sim” signifies the ImageNet-based Similarity Cache. Similarly, “TOCS sim” in Fig. 12 (b) stands for the Similarity Cache with TOCS’s text labels.

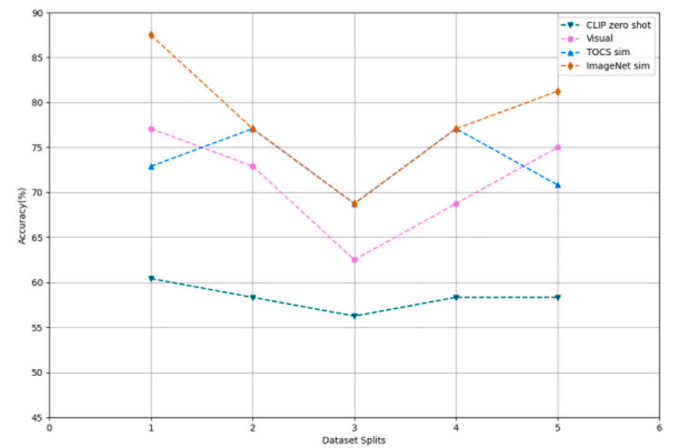
Fig. 12 (a) reveals the independent prediction accuracy of each cache on SODA. In the 1-shot and 2-shot settings, the “ImageNet sim” demonstrates the best accuracy among all compared approaches. It achieves 43% accuracy with only one training image, whereas the “SODA sim” reaches 32.45% in the same setting. This suggests that the 1000 ImageNet text labels provide a more useful distribution of image-text similarity features for each object class. However, for 4-shot setups and beyond, the Visual Cache surpasses all independent contributions in

prediction accuracy. This implies that with more visual features, the Visual Cache’s prediction accuracy increases faster than the Similarity Cache’s.

Fig. 12 (b) focuses on the 1-shot setting, where the ImageNet-based Similarity Cache again demonstrates the highest average accuracy across the five dataset splits. The difference in prediction accuracy on different splits can be attributed to randomly selecting one-shot learning images. Notably, in Split 1, “ImageNet sim” achieves an impressive 87.5% accuracy, compared to 73% for the “TOCS sim”. This substantial improvement, observed across all splits, underscores the effectiveness of leveraging open-source ImageNet text labels for enriching semantic



(a)



(b)

Fig. 12. Classification accuracy for independent caches on (a) SODA and (b) TOCS.



features and enhancing classification performance.

To further elucidate the distinct representations learned by each Cache, Fig. 13 employs the t-SNE dimensionality reduction tool [34] to visualize the features of test images from the SODA dataset. Fig. 13(a) visualizes the test image features  $I_{query}$  for Visual Cache prediction. Fig. 13 (b) displays the test similarity features  $SIM_{query\_imgNet}$  for Similarity Cache prediction in the proposed algorithm. Each distinct color represents one of the 14 temporary object categories in the SODA Test Set. Notably, compared to  $I_{query}$ , the proposed  $SIM_{query\_imgNet}$  exhibits superior intra-class clustering, evidenced by tighter clusters within each category. Furthermore, the inter-class boundaries, the demarcations between different categories, are visually more pronounced for  $SIM_{query\_imgNet}$ . This demonstrates the enhanced discriminative power of the proposed ImageNet-based Similarity Cache for distinguishing between various object classes.

#### 4.4.2. Different matching functions for similarity cache

One major difference of the proposed algorithm from TIP-X lies in its Similarity Cache Matching function, expressed in Eq. (9). This equation provides an effective approach to gauge the affinity between the cached similarity distribution  $SIM_{support\_imgNet}$  and the query similarity distribution  $I_{query}$ . In contrast, TIP-X relies on KL-divergence to measure the affinity between the query distribution and support distributions. To facilitate a direct comparison, this section utilizes Target Dataset text labels within the proposed algorithm's matching function, mimicking the approach adopted by TIP-X. Table 12 presents the comparative classification accuracy for both SODA and TOCS datasets when employing different matching functions. The proposed Eq. (9) indicates higher classification accuracy on average for both SODA and TOCS. In the 4-shot setup of SODA and Split 1 of TOCS, KL-divergence is slightly better than the proposed function. While the proposed function demonstrates overall effectiveness, surpassing the widely used KL-divergence, the performance gap between the two approaches is not substantial. Both functions adequately capture the affinity between the support and query distributions, suggesting that the choice may depend on specific dataset characteristics and task requirements.

#### 4.4.3. Study of hyperparameters

The proposed algorithm has three hyperparameters,  $\beta$  in Eq. (5),  $\alpha_1$  and  $\alpha_2$  in Eq. (14). The sharpness of the exponential function is controlled by  $\beta$  in Eq. (5). As Eq. (14) illustrates, the final prediction, represented by *Logit*, is a weighted sum of three components: CLIP zero-shot, Visual Cache, and Similarity Cache.  $\alpha_1$  and  $\alpha_2$  determine the respective weights of the Visual Cache and Similarity Cache predictions, which are rescaled to match the range of CLIP zero-shot. Their ratio defines the relative importance of each component. From Eq. (14), the

weight ratio of CLIP zero-shot, Visual Cache and Similarity Cache is 1:  $\alpha_1$ :  $\alpha_2$ . If  $\alpha_1$  or  $\alpha_2$  equals to 1.0, it means Visual Cache or Similarity Cache has the same importance with the result of CLIP zero-shot.

Table 13 presents the optimal hyperparameter values obtained through grid search on Validation Set to maximize accuracy. The sharpness ratio  $\beta$  exhibits a moderate increase from 28.04 to 36.00 as the number of learning shots increases from 1 to 16. The number of learning shots suggests limited impact on the selection of  $\beta$  value. The weight of Similarity Cache  $\alpha_2$  changes from 1.09 to 4.06 with different learning shots. The value of  $\alpha_2$  is consistently larger than 1, suggesting that Similarity Cache has greater importance compared to CLIP zero-shot. This is particularly evident in the 1-shot and 2-shot setups, where  $\alpha_2$  values of 3.07 and 4.06, respectively. It indicates that Similarity Cache predictions are three to four times more impactful than CLIP zero-shot. The weight of Visual Cache  $\alpha_1$  shows a significant increase from 9.5 in the 1-shot setup to 170.0 in the 16-shot setup. This trend implies the growing dominance of Visual Cache as the number of learning shots increases. The accumulating visual features within the Cache enhance its reliability and predictive power. In contrast, Similarity Cache shows limited ability to be improved by more learning shots.

#### 4.4.4. Study of text prompts

CLIP has learned a wide range of visual concepts directly from natural language. CLIP zero-shot classification is sensitive to the text prompts that describe the images. This section compares the CLIP zero-shot classification performance using different text prompts. Two types of text prompts are compared: the basic text prompts and handcrafted text prompts. Basic text prompt applies the template "This is a photo of X" by replacing "X" with object labels. Handcrafted text prompts are created for each object using dictionaries and Wikipedia, shown in Table 14. Fig. 14 shows CLIP zero-shot classification using text prompts "This is a photo of X" and handcrafted text prompts in Table 14. Handcrafted text prompts successfully rectify misclassifications predicted by basic text prompts. This improvement arises from the enriched contextual knowledge within the handcrafted prompts, enabling a more accurate matching with on-site objects. For example, the trolley is classified wrongly as "rebar" or "ladder" in the left part of Fig. 14. With handcrafted text prompts in Table 14, the function of "trolley" is described as "transport and carry heavy loads". The specific description contributes to the correct classification in the right part of Fig. 14. Similarly, for the photo of a bundle of buckets, the description of its shape as "round container" and its function of "catching, holding or carrying liquids or solids" contributes to the improved classification in the right part of Fig. 14. CLIP's prior knowledge may include low-level knowledge such as colors, shapes of visual features and high-level semantic information such as the purpose of use for a tool or construction

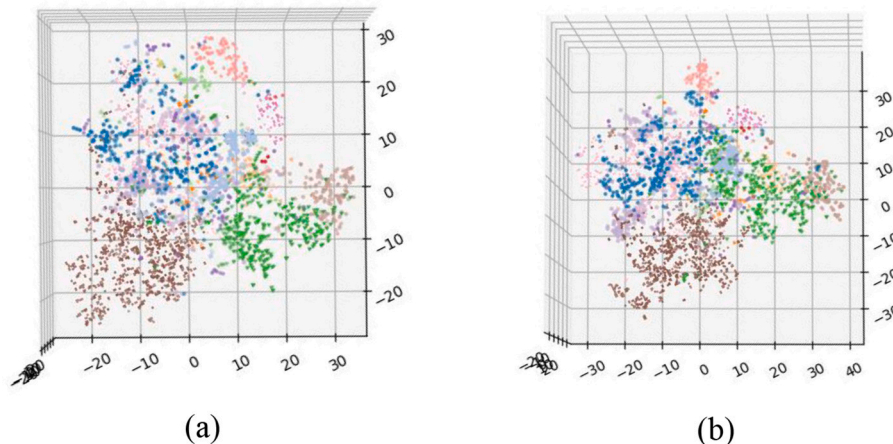


Fig. 13. Visualization of features for (a)  $I_{query}$  and (b)  $SIM_{query\_imgNet}$ .

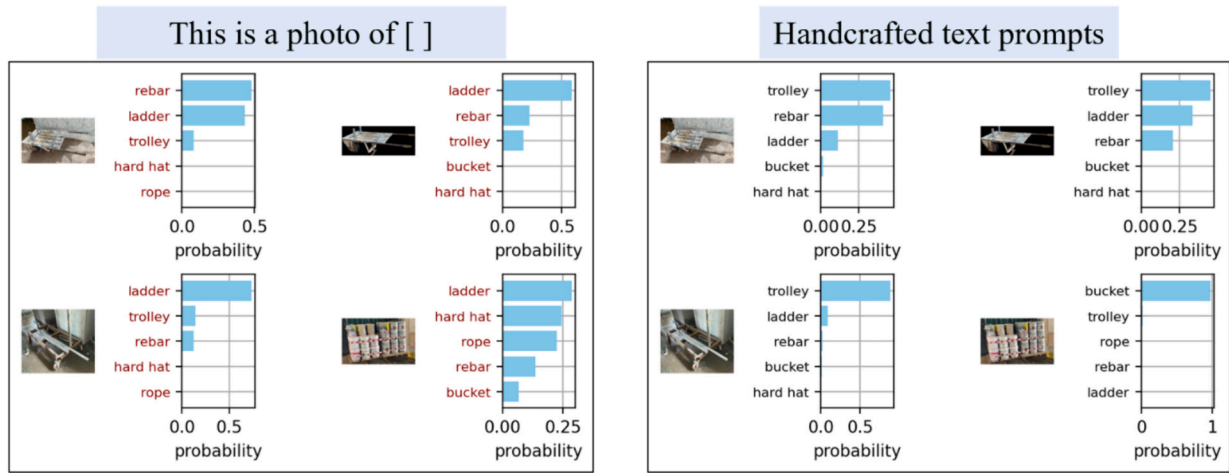


Fig. 14. Zero-shot classification with different text prompts.

material. Textual prompts with object related knowledge will significantly improve CLIP zero-shot performance.

## 5. Discussions, applications and future work

Due to the lack of open-source datasets for on-site object classification, the proposed algorithm is evaluated on modified SODA and TOCS. Regions of interest of original SODA images are cropped. The tight bounding box may lose the context for pixels around the on-site temporary objects. The lack of background context partially accounts for the relatively low accuracy of CLIP zero-shot prediction. Despite the proposed TOCS dataset having 22 categories of temporary objects, the total number of images is 90. The proposed algorithm could be evaluated with more comprehensive construction datasets in future.

The proposed algorithm adopts the basic text prompt “This is a photo of X”. In Section 4.4.4, CLIP has shown higher zero-shot classification accuracy for temporary objects with the help of detailed textual prompts. To further improve the CLIP-based few-shot learning ability, elaborate textual prompts can be explored.

The proposed algorithm introduces the open-source text labels for building Similarity Cache. The study focuses on text labels from ImageNet. Other text labels could be explored in future to provide inter-class similarity distribution for building Similarity Cache. The criteria for selecting these text labels will be useful.

Besides object class recognition, object locations are useful for robotic construction and construction management. The proposed few-shot classification algorithm can potentially be applied for few-shot object detection on construction sites. Object detection can be decomposed into two sub-problems: object localization and classification. Typical two-stage object detectors such as Faster R-CNN [35] extract region proposals in the first stage and classify them in the second stage. Region localization module in the first stage can be trained without knowing the object class. The proposed CLIP-based few-shot image classification algorithm can be integrated with a class-agnostic localization module for few-shot object detection for construction objects.

## 6. Conclusion

The recent success of vision-language foundation models like CLIP has built a strong capability for feature extraction and cross-modality correlation. To address the challenging task of few-shot temporary object recognition for robots and construction management, this paper proposes a CLIP-based few-shot image classification algorithm. The proposed algorithm does not require any training to finetune the weights of the neural network. Knowledge from few-shot images is learned by building Visual Cache and Similarity Cache. Compared with existing

CLIP-based few-shot algorithms, the proposed algorithm introduces open-source object names to build Similarity Cache. Specifically, text labels from ImageNet are used to build image-text similarity distribution as class prototypes. ImageNet-based Similarity Cache makes it practical for on-site learning without the input of well-defined object names.

To Validate the performance of the proposed algorithm, the object detection dataset SODA is modified for image classification. This study also proposes TOCS dataset with 22 different types of on-site objects. Experiments on SODA and TOCS demonstrate the superior performance of the proposed algorithm compared with all the baseline algorithms. While CLIP zero-shot achieves only 23.17% accuracy on SODA, the proposed algorithm achieves 73.09% accuracy with only 16 shots of learning images. With 1-shot learning, the proposed algorithm improves the classification accuracy from 58% to 83% on the proposed TOCS. The proposed algorithm surpasses the state-of-the-art (SOTA) algorithm TIP-X by more than 5% accuracy with 1-shot learning on both SODA and TOCS.

The contributions of this paper are twofold. First, the proposed algorithm introduces ImageNet-based Similarity Cache, enhancing the CLIP-based few-shot classification accuracy using publicly available open-source object labels. The proposed Similarity Cache matching function better evaluates the test feature with prototypes. Second, the TOCS dataset is proposed to evaluate the performance of the proposed algorithm. The SODA dataset for object detection is modified and accommodated for few-shot image classification task. This work illustrates the possible future work to improve on-site visual intelligence by utilizing VLM.

The limitation of this research has been identified as limited public dataset for validation and adoption of basic text prompts. Further research is needed to investigate elaborate text prompts for VLM-based few-shot learning. Besides ImageNet text labels, other effective open-source text prompts could be explored to further improve the performance. There are also opportunities to adopt such training-free learning algorithms for few-shot object detection (FSOD) on construction sites.

## CRedit authorship contribution statement

**Yuanchang Liang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Prahlad Vadakkepat:** Writing – review & editing, Supervision. **David Kim Huat Chua:** Writing – review & editing, Supervision, Resources. **Shuyi Wang:** Investigation, Data curation. **Zhigang Li:** Data curation. **Shuxiang Zhang:** Writing – review & editing.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## Appendix

### Summary of Notations

| Symbol                  | Explanation                                                                                                            | Dimension                 |
|-------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------|
| $\mathcal{S}_{support}$ | Support Set                                                                                                            |                           |
| $\mathcal{S}_{query}$   | Query Set                                                                                                              |                           |
| $N$                     | Number of object classes                                                                                               |                           |
| $C$                     | Number of channels for both image and text embeddings                                                                  |                           |
| $K$                     | Number of images per class                                                                                             |                           |
| $n$                     | Total number of images in the support set ( $n = K \times N$ )                                                         | $\mathbb{R}^{1 \times 1}$ |
| $M$                     | Number of object classes in ImageNet                                                                                   |                           |
| $I_i$                   | Image embedding                                                                                                        |                           |
| $T_i$                   | Text embedding                                                                                                         | $\mathbb{R}^{1 \times C}$ |
| $I_{support}$           | Image embeddings of the Support Set                                                                                    | $\mathbb{R}^{n \times C}$ |
| $I_{query}$             | Image embeddings of one query image from Query Set                                                                     | $\mathbb{R}^{1 \times C}$ |
| $T_{target}$            | Text embeddings of Target Dataset class labels                                                                         | $\mathbb{R}^{n \times C}$ |
| $T_{imgNet}$            | Text embeddings of ImageNet class labels                                                                               | $\mathbb{R}^{m \times C}$ |
| $SIM_{support\_target}$ | Cosine similarity distribution between support images and Target Dataset's labels                                      | $\mathbb{R}^{n \times N}$ |
| $SIM_{support\_imgNet}$ | Cosine similarity distribution between support images and ImageNet labels                                              | $\mathbb{R}^{n \times M}$ |
| $SIM_{query\_target}$   | Cosine similarity distribution between one query image and Target Dataset's labels                                     | $\mathbb{R}^{1 \times N}$ |
| $SIM_{query\_imgNet}$   | Cosine similarity distribution between one query image and ImageNet labels                                             | $\mathbb{R}^{1 \times M}$ |
| $A_{vis}$               | The affinity between query visual feature and cached support visual features in Visual Cache                           | $\mathbb{R}^{1 \times n}$ |
| $A_{sim}$               | The affinity between one query similarity distribution and cached support similarity distributions in Similarity Cache |                           |
| $L_{support}$           | One-Hot encoding of object class labels                                                                                | $\mathbb{R}^{n \times N}$ |
| $Logit_{zshot}$         | Prediction logits of CLIP Zero-shot                                                                                    |                           |
| $Logit_{vis}$           | Prediction logits of Visual Cache                                                                                      |                           |
| $Logit_{sim}$           | Prediction logits of Similarity Cache                                                                                  | $\mathbb{R}^{1 \times N}$ |
| $Logit$                 | Overall prediction logits                                                                                              |                           |

## References

- [1] S.K. Baduge, S. Thilakarathna, J.S. Perera, M. Arashpour, P. Sharafi, B. Teodosio, A. Shringi, P. Mendis, Artificial intelligence and smart vision for building and construction 4.0: machine and deep learning methods and applications, *Autom. Constr.* 141 (2022) 104440, <https://doi.org/10.1016/j.autcon.2022.104440>.
- [2] N. Melenbrink, J. Werfel, A. Menges, On-site autonomous construction robots: towards unsupervised building, *Autom. Constr.* 119 (2020) 103312, <https://doi.org/10.1016/j.autcon.2020.103312>.
- [3] J. Teizer, Status quo and open challenges in vision-based sensing and tracking of temporary resources on infrastructure construction sites, *Adv. Eng. Inform.* 29 (2015) 225–238, <https://doi.org/10.1016/j.aei.2015.03.006>.
- [4] S. Paneru, I. Jeelani, Computer vision applications in construction: current state, opportunities & challenges, *Autom. Constr.* 132 (2021) 103940, <https://doi.org/10.1016/j.autcon.2021.103940>.
- [5] X. Wu, D. Sahoo, S.C.H. Hoi, Recent advances in deep learning for object detection, *Neurocomputing* 396 (2020) 39–64, <https://doi.org/10.1016/j.neucom.2020.01.085>.
- [6] W.-Y. Chen, Y.-C. Liu, Z. Kira, Y.-C.F. Wang, J.-B. Huang, A closer look at few-shot classification, in: 7th International Conference on Learning Representations (ICLR 2019), New Orleans, LA, USA, 2019, <https://doi.org/10.48550/arXiv.1904.04232>.
- [7] L. Zhou, P. Cui, X. Jia, S. Yang, Q. Tian, Learning to select base classes for few-shot classification, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, Seattle, WA, USA, 2020, pp. 4623–4632, <https://doi.org/10.1109/CVPR42600.2020.00468>.
- [8] R. Zhang, W. Zhang, R. Fang, P. Gao, K. Li, J. Dai, Y. Qiao, H. Li, Tip-adapter: Training-free adaption of CLIP for few-shot classification, in: S. Avidan, G. Brostow, M. Cissé, G.M. Farinella, T. Hassner (Eds.), *Computer Vision – ECCV 2022*, ECCV 2022, Lecture Notes in Computer Science, Springer, Cham, Berlin, Heidelberg, 2022, pp. 493–510, [https://doi.org/10.1007/978-3-031-19833-5\\_29](https://doi.org/10.1007/978-3-031-19833-5_29).
- [9] A. Radford, J.W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, Learning transferable visual models from natural language supervision, in: Proceedings of the 38th International Conference on Machine Learning, PMLR, 2021, pp. 8748–8763, in: <https://proceedings.mlr.press/v139/radford21a.html> (accessed April 9, 2024).
- [10] J. Deng, W. Dong, R. Socher, L.-J. Li, Kai Li, Li Fei-Fei, ImageNet: A large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, IEEE, Miami, FL, USA, 2009, pp. 248–255, <https://doi.org/10.1109/CVPR.2009.5206848>.
- [11] J. Yang, M.-W. Park, P.A. Vela, M. Golparvar-Fard, Construction performance monitoring via still images, time-lapse photos, and video streams: now, tomorrow, and the future, *Adv. Eng. Inform.* 29 (2015) 211–224, <https://doi.org/10.1016/j.aei.2015.01.011>.
- [12] Z.-Q. Zhao, P. Zheng, S.-T. Xu, X. Wu, Object detection with deep learning: a review, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (2019) 3212–3232, <https://doi.org/10.1109/TNNLS.2018.2876865>.
- [13] W. Fang, L. Ding, B. Zhong, P.E.D. Love, H. Luo, Automated detection of workers and heavy equipment on construction sites: a convolutional neural network approach, *Adv. Eng. Inform.* 37 (2018) 139–149, <https://doi.org/10.1016/j.aei.2018.05.003>.
- [14] X. Luo, H. Li, D. Cao, F. Dai, J. Seo, S. Lee, Recognizing diverse construction activities in site images via relevance networks of construction-related objects detected by convolutional neural networks, *J. Comput. Civ. Eng.* 32 (2018) 04018012, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000756](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000756).
- [15] D. Wang, X. Wang, B. Ren, J. Wang, T. Zeng, D. Kang, G. Wang, Vision-based productivity analysis of cable crane transportation using augmented reality-based synthetic image, *J. Comput. Civ. Eng.* 36 (2022) 04021030, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000994](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000994).

- [16] S. Kumar, H. Gupta, D. Yadav, I.A. Ansari, O.P. Verma, YOLOv4 algorithm for the real-time detection of fire and personal protective equipments at construction sites, *Multimed. Tools Appl.* 81 (2022) 22163–22183, <https://doi.org/10.1007/s11042-021-11280-6>.
- [17] Y. Song, T. Wang, P. Cai, S.K. Mondal, J.P. Sahoo, A comprehensive survey of few-shot learning: evolution, applications, challenges, and opportunities, *ACM Comput. Surv.* 55 (2023) 1–40, <https://doi.org/10.1145/3582688>.
- [18] X. Li, X. Yang, Z. Ma, J.-H. Xue, Deep metric learning for few-shot image classification: a review of recent developments, *Pattern Recogn.* 138 (2023) 109381, <https://doi.org/10.1016/j.patcog.2023.109381>.
- [19] S.X. Hu, D. Li, J. Stuhmer, M. Kim, T.M. Hospedales, Pushing the limits of simple pipelines for few-shot learning: External data and fine-tuning make a difference, in: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, New Orleans, LA, USA, 2022, pp. 9058–9067, <https://doi.org/10.1109/CVPR52688.2022.00886>.
- [20] H. Wang, H. Zhao, B. Li, Bridging multi-task learning and meta-learning: Towards efficient training and effective adaptation, in: M. Meila, T. Zhang (Eds.), *Proceedings of the 38th International Conference on Machine Learning*, PMLR, 2021, pp. 10991–11002, in: <https://proceedings.mlr.press/v139/wang21ad.html> (Accessed 9 April 2024).
- [21] L. Khacef, V. Gripon, B. Miramond, GPU-based self-organizing maps for post-labeled few-shot unsupervised learning, in: H. Yang, K. Pasupa, A.C.-S. Leung, J. T. Kwok, J.H. Chan, I. King (Eds.), *Neural Information Processing*, Springer International Publishing, Cham, 2020, pp. 404–416, [https://doi.org/10.1007/978-3-030-63833-7\\_34](https://doi.org/10.1007/978-3-030-63833-7_34).
- [22] X. Zhong, C. Gu, W. Huang, L. Li, S. Chen, C.-W. Lin, Complementing representation deficiency in few-shot image classification: A meta-learning approach, in: 2020 25th International Conference on Pattern Recognition (ICPR), IEEE, Milan, Italy, 2021, pp. 2677–2684, <https://doi.org/10.1109/ICPR48806.2021.9412416>.
- [23] J. Xu, J.-F. Ton, H. Kim, A.R. Kosiorek, Y.W. Teh, MetaFun: Meta-Learning with iterative functional updates, in: *Proceedings of the 37th International Conference on Machine Learning*, PMLR, 2020, pp. 10617–10627, in: <https://proceedings.mlr.press/v119/xu20i.html> (accessed April 9, 2024).
- [24] M. Patacchiola, J. Turner, E.J. Crowley, M. O'Boyle, A. Storkey, Bayesian meta-learning for the few-shot setting via deep kernels, in: *Advances in Neural Information Processing Systems (NeurIPS 2020)*, Curran Associates, Inc., Vancouver, Canada, 2020, pp. 16108–16118, <https://doi.org/10.48550/ARXIV.1910.05199>.
- [25] C. Finn, P. Abbeel, S. Levine, Model-agnostic meta-learning for fast adaptation of deep networks, in: *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, PMLR, Sydney, NSW, Australia, 2017, pp. 1126–1135, in: <https://proceedings.mlr.press/v70/finn17a.html> (accessed April 9, 2024).
- [26] G. Koch, R. Zemel, R. Salakhutdinov, Siamese neural networks for one-shot image recognition, in: *ICML Deep Learning Workshop*, 2015, <https://api.semanticscholar.org/CorpusID:13874643> (accessed April 9, 2024).
- [27] Z. Cui, Q. Wang, J. Guo, N. Lu, Few-shot classification of façade defects based on extensible classifier and contrastive learning, *Autom. Constr.* 141 (2022) 104381, <https://doi.org/10.1016/j.autcon.2022.104381>.
- [28] Y. Xu, Y. Bao, Y. Zhang, H. Li, Attribute-based structural damage identification by few-shot meta learning with inter-class knowledge transfer, *Struct. Health Monit.* 20 (2021) 1494–1517, <https://doi.org/10.1177/1475921720921135>.
- [29] J. Kim, S. Chi, A few-shot learning approach for database-free vision-based monitoring on construction sites, *Autom. Constr.* 124 (2021) 103566, <https://doi.org/10.1016/j.autcon.2021.103566>.
- [30] V. Udandarao, A. Gupta, S. Albanie, SuS-X: Training-free name-only transfer of vision-language models, in: 2023 IEEE/CVF International Conference on Computer Vision (ICCV), IEEE, Paris, France, 2023, pp. 2725–2736, <https://doi.org/10.1109/ICCV51070.2023.00257>.
- [31] X. Zhu, R. Zhang, B. He, A. Zhou, D. Wang, B. Zhao, P. Gao, Not all features matter: Enhancing few-shot CLIP with adaptive prior refinement, in: 2023 IEEE/CVF International Conference on Computer Vision (ICCV), IEEE, Paris, France, 2023, pp. 2605–2615, <https://doi.org/10.1109/ICCV51070.2023.00246>.
- [32] A. Krizhevsky, G. Hinton, Learning Multiple Layers of Features from Tiny Images, University of Toronto, 2009, <https://www.cs.toronto.edu/~kriz/cifar.html> (accessed April 9, 2024).
- [33] R. Duan, H. Deng, M. Tian, Y. Deng, J. Lin, SODA: a large-scale open site object detection dataset for deep learning in construction, *Autom. Constr.* 142 (2022) 104499, <https://doi.org/10.1016/j.autcon.2022.104499>.
- [34] L. van der Maaten, G. Hinton, Visualizing data using t-SNE, *J. Mach. Learn. Res.* 9 (2008) 2579–2605, <http://jmlr.org/papers/v9/vandermaaten08a.html> (accessed April 9, 2024).
- [35] S. Ren, K. He, R. Girshick, J. Sun, Faster R-CNN: towards real-time object detection with region proposal networks, *IEEE Trans. Pattern Anal. Mach. Intell.* 39 (2017) 1137–1149, <https://doi.org/10.1109/TPAMI.2016.2577031>.